VectorForge

Storage engine desacoplado para vetores e payloads, com busca 100% plugável — side project de aprendizado em Rust, motivado por uma lacuna real identificada durante o trabalho de paralelismo da faculdade.

Contexto: autor com C avançado, sem experiência prévia em Rust, trabalhando full-time na Qdrant. Projeto pensado para ser desenvolvido em par com Claude Code, no ritmo de side project.

1. Motivação e o gap identificado

Bancos vetoriais atuais (Qdrant, Milvus, Weaviate) acoplam fortemente storage e algoritmo de busca — o índice (HNSW, IVF) é parte do motor, não um plugin. Bibliotecas de índice puro (Faiss, hnswlib, usearch) resolvem a busca mas não oferecem storage persistente, CRUD ou gerenciamento de memória. Formatos columnar como Lance cobrem parte do meio-termo, mas ainda vêm com API própria de banco. O gap real: uma camada de storage genérica — vetores + payload, com controle explícito de memória (RAM vs. disco) — que não impõe nenhum algoritmo de busca, deixando isso inteiramente a cargo de quem usa a engine.

Diferencial real do autor: experiência direta na Qdrant expõe onde bancos vetoriais travam na prática (ex.: compaction do RocksDB causando latency spikes, que motivou a Qdrant a construir seu próprio key-value store). Esse conhecimento de primeira mão é o que dá direção ao design, mais do que a ambição de "reinventar o banco vetorial".

2. Avaliação honesta do stack técnico

Rust é a escolha certa para esse projeto especificamente por causa do mmap: em C++, um bug de ponteiro sobre uma região mmap compartilhada entre threads é silencioso e pode corromper dados sem crash imediato. Em Rust, o borrow checker força a explicitar quem tem acesso mutável a essa região — o preço é uma curva de aprendizado dura logo de início, porque justamente as partes mais interessantes do projeto (mmap + concorrência) são as que exigem unsafe, ou seja, a parte do Rust onde o compilador para de ajudar. Isso não invalida a escolha, mas é bom entrar sabendo que o começo vai ser mais lento do que "aprender Rust" em abstrato — vai ser "aprender Rust unsafe e concorrente" ao mesmo tempo em que se aprende a linguagem.

Avaliação crate por crate

memmap2	Escolha correta e sem alternativa melhor. API pequena, bem documentada, é o padrão de facto para mmap em Rust. Baixo risco.
rayon	Excelente ponto de entrada em paralelismo Rust vindo de OpenMP/C: par_iter() substitui #pragma omp parallel for quase 1 para 1 conceitualmente. Risco baixo, curva de aprendizado suave — comece por aqui.
nix (madvise/mlock)	Correto tecnicamente, mas é otimização fina de kernel. Adiar para depois do MVP funcionar — mexer nisso cedo demais consome tempo sem validar a ideia central.
sled	Cautela: o projeto está oficialmente em beta há anos, sem previsão firme de 1.0, e o ritmo de manutenção é lento. Funciona, mas é risco de dependência estagnada.
fjall (alternativa a sled)	Mais ativo em 2026, API BTreeMap-like, 100% Rust seguro, particionamento (column families) e transações opcionais. Melhor escolha hoje que sled para o payload store.

redb (outra alternativa)	Também ativo, ACID completo, API mais simples que fjall, inspirado em LMDB. Boa opção se preferir uma API mais parecida com uma tabela única.
bincode	Serialização simples e madura — comece por aqui para o payload.
rkyv (zero-copy)	Tecnicamente superior (zero-copy deserialization), mas tem curva de aprendizado própria (lifetimes, versionamento de schema, macros). Trocar o bincode por rkyv só depois que o MVP funcionar e um benchmark mostrar que a desserialização é gargalo real — não antes disso.

Recomendação de ajuste no stack original

Trocar sled por fjall ou redb para o payload store, e tratar nix (madvise/mlock) e rkyv como otimizações de uma fase posterior, não do MVP. O restante do stack (memmap2, rayon, bincode) está correto como está.

3. Arquitetura

Dois segmentos de dados desacoplados, seguindo o mesmo padrão storage/índice discutido no trabalho da faculdade — só que agora persistido em disco via mmap, em vez de só em RAM.

Vector Matrix Segment

Arquivo binário plano com vetores densos, layout fixo (mesma ideia do array achatado: offset do vetor i = header_size + i * D * sizeof(f32)). Mapeado via mmap, exposto ao código do usuário como um slice de memória (&[f32]) — não como uma API de query.

Payload Store Segment

Key-value separado (fjall/redb), ID do vetor como chave. Só é acessado depois que o algoritmo de busca do usuário já decidiu quais IDs são "vencedores" — mantém o caminho de busca (hot path) livre de I/O de metadado.

Memory Sandbox (o ponto central do produto)

A engine não expõe nenhuma função de busca. Ela expõe: (1) um slice de memória mapeado com todos os vetores, (2) uma função de leitura de payload por ID, (3) primitivos de hint de memória (madvise/mlock, fase posterior). O usuário escreve seu próprio brute-force, LSH, HNSW ou IVF em cima disso, usando rayon para paralelizar como quiser — a engine nunca decide como buscar, só como acessar os dados.

4. Desafios técnicos reais (o que o roadmap original não cobre)

Unsafe Rust + mmap + concorrência, tudo ao mesmo tempo

mmap entrega um ponteiro cru. Expor isso como um slice seguro (&[f32]) para múltiplas threads exige unsafe cuidadosamente encapsulado, provando pro compilador (via invariantes que você garante manualmente) que aquela região não vai ser escrita e lida ao mesmo tempo de forma inconsistente. Essa é a parte mais difícil do projeto tecnicamente, e o roadmap original a trata como um detalhe do Mês 2 — na prática, é provável que consuma boa parte do tempo total.

Escrita concorrente e deleção (fora do MVP, mas precisa ser uma decisão consciente)

O roadmap original só cobre escrita em lote inicial e leitura. Updates e deletes num arquivo de vetores contíguo são o problema que já apareceu nesta conversa (compaction do RocksDB). Decisão recomendada para o MVP: engine append-only e read-only após a carga inicial — deixar update/delete explicitamente fora de escopo, documentado como tal, em vez de simplesmente não mencionado.

Crash safety

Um crash no meio de uma escrita no Vector Matrix Segment pode deixar o arquivo com um vetor parcialmente escrito. Para o MVP (append-only, carga em lote única), mitigar com um flag de "escrita

completa" gravado por último (ex.: um contador de vetores válidos atualizado só após o mmap inteiro ser sincronizado com msync). Uma WAL de verdade fica para uma fase posterior.

Validação e benchmark justo

Comparar contra Faiss (IndexFlatL2) com o mesmo dataset e mesma contagem de threads dá credibilidade ao resultado final — sem isso, "funcionou" não distingue se o motor é competitivo ou só correto.

5. Roadmap realista

Ajustado para: aprendizado de Rust do zero, ritmo de side project (poucas horas por semana), trabalho em par com Claude Code para acelerar a parte mecânica e liberar tempo para entender o que está sendo escrito, não só copiar. O roadmap original de 3 meses vira 5-6 meses em ritmo de side project — isso é reflexo do escopo real, não de falta de capacidade.

Fase 0 — Fundamentos de Rust (2-3 semanas)

- Ownership, borrow checker, lifetimes — via Rust Book ou Rustlings, antes de tocar em unsafe
- Escrever algo pequeno e completo sem unsafe (ex.: CLI simples) para pegar o fluxo da linguagem
- Objetivo desta fase: errar contra o borrow checker em código descartável, não no projeto real

Fase 1 — CRUD básico do Vector Matrix Segment (3-4 semanas)

- Definir layout binário do arquivo (header: N, D, versão; corpo: vetores)
- Escrever função de escrita em lote (sem mmap ainda, I/O direto é suficiente aqui)
- Trocar para memmap2: mapear o arquivo, ler vetor por ID calculando o offset
- Testar com um dataset pequeno gerado aleatoriamente antes de qualquer coisa maior

Fase 2 — Memory Sandbox + paralelismo (3-4 semanas)

- Expor o slice de vetores mapeado para código de busca do usuário
- Escrever a primeira função de busca por cima da sandbox: brute-force sequencial
- Paralelizar com rayon (par_iter/par_chunks), variando 1, 2, 4, 8 threads
- Aqui mora o trabalho pesado de unsafe — reservar tempo extra nesta fase

Fase 3 — Payload store (2-3 semanas)

- Integrar fjall ou redb para o payload, indexado pelo mesmo ID do vetor
- Serializar payloads com bincode
- Testar o pipeline completo: busca decide IDs vencedores -> payload é lido só no final

Fase 4 — Validação e benchmark (2 semanas)

- Rodar o mesmo dataset e queries contra Faiss (IndexFlatL2) como referência
- Medir tempo de busca por número de threads, comparar escalabilidade
- Documentar limitações conhecidas (append-only, sem deleção, sem crash recovery robusto)

Fase 5 (opcional, se o interesse continuar) — Segundo algoritmo de busca

- Implementar um segundo algoritmo (LSH ou HNSW simplificado) sobre a mesma sandbox
- Validar que a promessa central do projeto se sustenta: trocar o algoritmo sem tocar no storage

Marco de validação da ideia (não do produto): ao final da Fase 2, já é possível responder se a arquitetura de sandbox de memória funciona na prática. Esse é o ponto certo para decidir se vale continuar até um projeto mais completo ou se o aprendizado obtido já cumpriu o objetivo principal.

6. Como usar Claude Code sem perder o aprendizado

O risco de programar em par com um agente ao aprender uma linguagem nova é aceitar código unsafe sem entender por que ele é seguro — justamente a parte mais delicada deste projeto. Formas de mitigar isso:

- Pedir para Claude Code explicar cada bloco unsafe linha a linha antes de aceitar, especialmente nas Fases 1 e 2
- Escrever a primeira versão de cada função nova por conta própria, mesmo que errada, antes de pedir revisão ou correção — isso é o que fixa o aprendizado
- Usar o agente para acelerar o que é mecânico e já compreendido (boilerplate, testes, parsers de arquivo) e não para as decisões de design da sandbox de memória

7. Riscos do projeto

Tempo	Side project ao lado de trabalho full-time tende a andar mais devagar que o planejado — o roadmap já assume isso, mas vale revisar o ritmo a cada fase.
Escopo	Tentação de generalizar cedo demais (suportar todo tipo de índice, todo tipo de payload) antes de validar o caso mais simples. Resistir até a Fase 4.
Concorrência de mercado	LanceDB e Zvec (Alibaba, 2026) ocupam parte do mesmo espaço. Não invalida o projeto como aprendizado ou portfólio, mas reduz a chance de ineditismo caso a meta evolua para produto real.